

DNS Resolver with Caching, Security, and Logging

Project Report

Submitted By

Muhammad Abser | 23K0030

Muhammad Ali | 23K0052

FAST School of Computing

National University of Computer & Emerging Sciences

Abstract

This project implements a small DNS forwarding system with TTL-aware caching, rate limiting, query randomization, and DNSSEC-aware response checks. It is designed to demonstrate practical resolver behavior, measurable performance benefits from caching, and basic defenses against cache poisoning and request flooding.

1. Motivation

The project was selected to study how a practical DNS resolver can be improved with caching and basic security controls while remaining simple enough to build, test, and explain end to end. DNS is a core internet service, so even a small prototype is useful for demonstrating reliability, latency reduction, and attack mitigation techniques in a controlled setting.

2. Overview

2.1 Significance of the Project

This project is significant because it combines resolver behavior, cache design, logging, and security hardening in one system. The work is practical because it runs as a local UDP service and can be benchmarked directly, and it is educational because each safeguard is implemented in a transparent way rather than hidden in a black-box framework.

2.2 Description of the Project

The system is a DNS forwarding server written in Python. It listens for client queries, forwards them to an upstream resolver, stores responses in an in-memory TTL cache, and returns cached answers when valid entries already exist. The server adds cache-poisoning defenses through randomized source ports and 0x20 query-name encoding, rate-limits requests per source IP, and logs each query for analysis.

2.3 Background of the Project

The design is informed by common DNS implementations and resolver security practices. BIND9 and dnspython provide the basis for standard DNS handling, while DNSSEC references such as RFC 4033 describe authenticity and integrity validation. The project also reflects the broader lesson from cache poisoning attacks that DNS responses must be validated carefully before they are trusted.

2.4 Project Category

This is a product-based implementation project with a strong research and evaluation component.

3. Features / Scope / Modules

1. Recursive DNS forwarding with upstream query handling and structured response parsing.
2. TTL-aware cache that improves repeated-query latency and expires stale entries automatically.
3. Per-client rate limiting to reduce abuse and deny simple flooding behavior.
4. Cache-poisoning hardening using 0x20 randomized casing and randomized source ports.
5. DNSSEC-aware validation checks for signed data, helping avoid blind trust in unsigned answers.
6. CSV logging that records query name, type, cache hit status, response code, and latency.
7. Benchmark mode that compares cold and warm lookups to quantify caching gains.

4. Project Planning

Week 1 focused on setting up the Python environment and exploring DNS libraries. Week 2 implemented query forwarding and response parsing. Week 3 added TTL caching and benchmarking support. Week 4 added security checks and rate limiting. Week 5 was reserved for integration testing, logging, and report preparation.

Team responsibilities:

- Muhammad Abser (23K0030): cache-poisoning defenses, rate limiting, DNSSEC-aware checks, report integration.
- Muhammad Ali (23K0052): query forwarding, caching, logging, benchmarking, and validation runs.

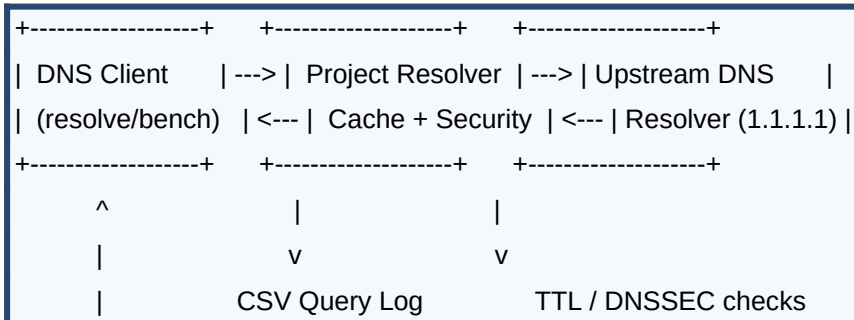
5. Project Feasibility

- Technical feasibility: the project is implementable with standard Python networking libraries and a public upstream DNS resolver.
- Economic feasibility: the software is low-cost and runs on a normal laptop or lab PC without special hardware.
- Schedule feasibility: the scope is small enough to complete in a short academic timeline while still supporting a meaningful demonstration.

6. Hardware and Software Requirements

- Hardware: a standard PC or laptop with network access.
- Operating system: Windows, Linux, or macOS; the implementation was tested in a Windows workspace.
- Software: Python 3.13, dnspython, and CSV-compatible tooling for log analysis.
- Optional tools: a local packet sniffer or Wireshark for observing query behavior.

7. Diagrammatic Representation of the Overall System



8. Results from Implementation

The current query log contains 39 recorded DNS transactions. Cache hits account for 28 queries (71.8%), while cache misses account for 11 queries. The average latency of cached responses is 0.993 ms, compared with 151.881 ms for cache misses. The observed latency range is 0.621 ms to 529.749 ms.

- No REFUSED responses were recorded in the available log slice.
- No SERVFAIL responses were recorded in the available log slice.
- Most frequently queried name: example.com..
- Most frequently queried type: A.

9. References

1. RFC 4033, DNS Security Introduction and Requirements.
2. BIND 9 Administrator Reference Manual.
3. dnspython documentation.
4. RFC 5358, Preventing DNS-Induced Denial-of-Service Attacks.
5. Kaminsky, D. (2008), DNS cache poisoning and related resolver attack discussions.
6. Ubuntu DNS documentation and resolver deployment guides.

10. Conclusion

The project demonstrates a functional DNS resolver prototype with measurable caching benefits and multiple defensive mechanisms. It is suitable as an academic project because the implementation is small enough to understand, yet rich enough to show real networking, security, and performance concepts in practice.